Learning Tiers for Long-Distance Processes

Magdalena Markowska

University of Surrey

Long-distance phonological dependencies, such as vowel harmony, can be modeled as Output Tier-based Strictly Local (OTSL) functions (Burness et al., 2021). However, algebraic analysis reveals that some of these processes also admit an input-oriented characterization (Lambert and Heinz, 2024). Here, we turn this theoretical insight into a learning strategy: we adapt an existing output-tier learner (Burness and McMullin, 2019) to infer input-tier content from sparse data using feature representations. The learned tier then determines the structure of an input-oriented transducer, whose outputs can be inferred separately. Experiments on Turkish, Kikuria, Finnish, and Yakut show that the learner successfully identifies conservative tiers by eliminating irrelevant intervening elements while retaining others when the available data do not provide sufficient evidence for their removal.

1 Long-distance but tier-local: Turkish vowel harmony

Turkish vowel harmony requires suffix vowels to agree in backness with the preceding vowel, regardless of how many consonants intervene. High suffix vowels also exhibit rounding harmony.

Backness harmony				
Nom.sg	Gen.sg	Nom.pl	Gen.pl	Gloss
ip	ip-in	ip-ler	ip-ler-in	*rope*
ev	ev-in	ev-ler	ev-ler-in	*house*
kɯz	kɯz-ɯn	kɯz-lar	kɯz-lar-ɯn	*girl*
sap	sap-ɯn	sap-lar	sap-lar-ɯn	*stalk*
Backness and rounding harmony				
jyz	jyz-yn	jyz-ler	jyz-ler-in	*face*
pul	pul-un	pul-lar	pul-lar-ɯn	*stamp*
son	son-un	son-lar	son-lar-ɯn	*end*
kœj	kœj-yn	kœj-ler	kœj-ler-in	*village*

The examples above show that Turkish vowels (targets and triggers of harmony) are separated by either one or two consonants (the interveners). In principle, there can be arbitrarily many of those interveners, which makes it impossible to describe such dependencies with a fixed memory window of k adjacent symbols. A common solution to this problem is to project only the symbols relevant to a tier and ignore the intervening material.

For example, a string $VCCCV$ becomes VV on a vowel tier: a local relation on the tier represents a nonlocal relation on the full string. In Turkish backness harmony, compare [ip-ler] and [pul-lar]: the suffix vowel agrees in backness with the preceding vowel, regardless of intervening consonants. Rounding harmony is slightly different. Consider [pul-un] and [pul-lar-ɯn]: in the first form, the high suffix vowel is rounded after /u/, whereas in the second, the intervening nonhigh vowels /E/ interrupts rounding harmony, and the final high suffix vowel is unrounded. Thus, different harmony processes may require different distinctions on the tier. Part of learning problem addressed here is to determine which input elements should be retained on the tier and which can be removed, from limited evidence.

2 Harmony as an output-oriented process

This relationship between adjacent vowels on the tier provides the basis for an output-oriented analysis of Turkish harmony. Under such analysis, the backness of a suffix vowel is determined by the preceding vowel on the output tier. The process can therefore be characterized as an Output Tier-based Strictly Local (OTSL) function for $k = 2$, where the immediately preceding output-tier vowel provides the relevant context (Chandlee et al., 2015; Burness et al., 2021).

For the backness harmony illustration, let e and a stand for fully specified front and back vowels, respectively; E is an underspecified nonhigh suffix vowel, I an underspecified high suffix vowel. The transducer below has two states that track whether the most recently output vowel is front or back. These states determine the realization of underspecified suffix vowels.

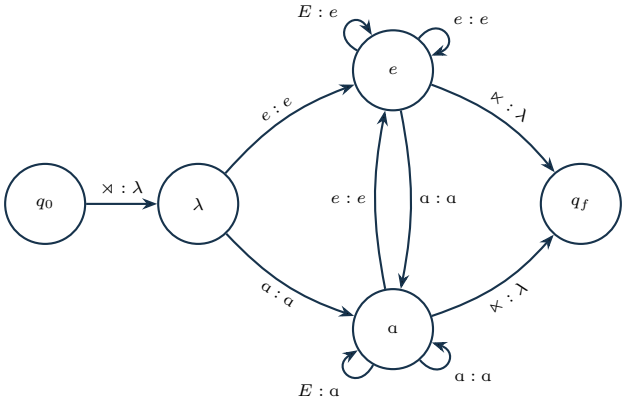


Figure 1: A 2-TOSL transducer for backness harmony.

3 Algebraic approach: action!

Lambert and Heinz (2023) characterize total input strictly local functions algebraically, and Lambert and Heinz (2024) show how some phonological processes commonly described as output-oriented can be reanalyzed through the transition semigroups of their minimal onward subsequential transducers. While strictly local descriptions specify a bounded window of adjacent symbols, the algebraic approach examines how input symbols affect transducer states. In Turkish harmony, this allows us to capture the role of intervening consonants, which leave the harmonic state unchanged regardless of how many occur.

The transition semigroup is built from the actions of input symbols, that is, the changes they induce in the transducer's state space. Given states $Q = \langle q_F, q_B \rangle$, the action induced by an input symbol $\sigma \in \Sigma$ is the tuple $\langle q_F \star \sigma, ; q_B \star \sigma \rangle$, where $q \star \sigma$ is the state reached from q upon reading σ.

Actions describe changes to the state space, independently of the outputs produced by the transducer. Different input symbols may induce the same action. In particular, symbols that leave every state unchanged induce the identity action **1**. Actions can be combined through composition: the product xy is obtained by first applying x and then y. The set of actions generated by input symbols under composition forms the transition semigroup.

For Turkish backness, we can construct a minimal transducer with two relevant control states representing the last element on the vowel tier, ${}^1q_F$ and ${}^2q_B$. Reading an input e when in the back state (${}^2q_B$) resets the machine to the front state. Similarly, reading a when in the front state resets it to the back state. Crucially, E, I, and C are *neutral* elements even though their *outputs* are not identical to the inputs. They are neutral because they do not cause transition out of state.

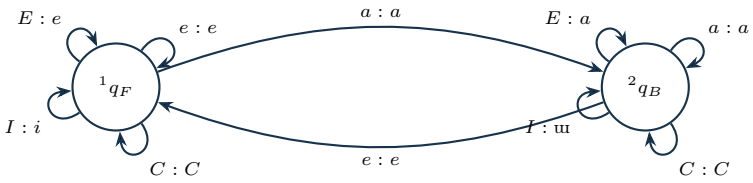


Figure 2: Minimal onward transducer capturing backness harmony in Turkish.

3.1 The actions and the multiplication table

There are two states, numbered $1 = q_F$ and $2 = q_B$. Each action is a two-tuple giving the destination from states 1 and 2:

$$1 = \langle 1, 2 \rangle \quad (E, I, C), \qquad f = \langle 1, 1 \rangle \quad (e), \qquad b = \langle 2, 2 \rangle \quad (a).$$

For example, $fb = b$: reading a back vowel after a front vowel overrides the front setting. By contrast, $f1 = f$: a consonant, or an underspecified vowel in this illustration, does not update the memory. The three actions make up the transition semigroup $S = \{1, f, b\}$.

$\cdot$	1	f	b
1	1	f	b
f	f	f	b
b	b	f	b

The multiplication table shows that each action is *idempotent* ($x^2 = x$): applying the same action twice has the same effect as applying it once. Moreover, the order of actions matters: $fb = b$ and $bf = f$, meaning that the most recently encountered specified vowel determines the harmonic state, while transparent segments leave it unchanged.

3.2 Algebraic classification of Turkish harmony

Green's relations compare what can be obtained by placing other actions *before*, *after*, or *on both sides* of a fixed action.

$$\begin{aligned} \mathcal{L}: &\quad Sx && \text{prefixes before } x \quad \text{(columns of the table)},\\ \mathcal{R}: &\quad xS && \text{suffixes after } x \quad \text{(rows)},\\ \mathcal{J}: &\quad SxS && \text{both sides of } x. \end{aligned}$$

For instance, $Sf = \{1f, ff, bf\} = \{f\}$, whereas $fS = \{f1, ff, fb\} = \{f, b\}$. Two actions are related if their corresponding sets are equal; a relation is *trivial* if no two distinct actions are related.

x	Sx (before)	xS (after)	SxS (both)
1	$\{1, f, b\}$	$\{1, f, b\}$	$\{1, f, b\}$
f	$\{f\}$	$\{f, b\}$	$\{f, b\}$
b	$\{b\}$	$\{f, b\}$	$\{f, b\}$

The columns yield different sets, so S is *$\mathcal{L}$-trivial*. The f and b rows yield the same set, so S is *not $\mathcal{R}$-trivial*. For $\mathcal{J}$, f and b share the smallest two-sided ideal $\{f, b\}$, the *minimal $\mathcal{J}$-class*.

A semigroup is *definite* (**D**) when it is $\mathcal{L}$-trivial and all idempotents lie in the minimal $\mathcal{J}$-class; *reverse definite* (**K**) uses $\mathcal{R}$-triviality instead. Here the identity 1 is idempotent but lies *outside* $\{f, b\}$. Thus S is not ordinarily definite. Once the identity action of neutral material is set aside, however, the remaining semigroup $\{f, b\}$ is definite. The mapping therefore has a *tier-based definite* analysis, $[\![\mathbf{D}]\!]_T$ (Lambert, 2023; Lambert and Heinz, 2024).

Learning consequence. If a process admits an input-tier-based characterization, we need not learn an output-oriented transducer. Instead, we can identify the relevant input symbols, use them to construct an input-tier transducer, and infer its transition outputs using learning methods developed for local processes. The key challenge, then, is to determine *which input symbols belong on the tier*.

4 Learning input-tier content

The algebraic analysis motivates an input-oriented approach to learning, but it does not tell us which input symbols belong on the tier. Burness and McMullin (2019) address the corresponding problem for output tiers: their learner starts with the full output alphabet and eliminates symbols that cannot reliably predict subsequent behavior. We adapt this approach to input tiers, comparing prefixes that share the same most recent tier symbol to determine whether they exhibit the same subsequent behavior.

To make this comparison, we need to establish how much of the output is already determined by an input prefix. For a function $f : \Sigma^* \to \Delta^*$, the *prefix function* is

$$f^p(u) = \mathrm{lcp}\{f(uv) : v \in \Sigma^*\}.$$

Here lcp denotes the longest common prefix. Thus, $f^p(u)$ is the output prefix shared by all possible continuations of the input prefix u. When another input symbol x is read, its *output contribution* is

$$\mathrm{cont}_f(u, x) = f^p(u)^{-1} f^p(ux),$$

where $r^{-1}s$ removes the prefix r from s. The contribution made at the end of a word can be defined as follows:

$$\mathrm{cont}_f(u, \ltimes) = f^p(u)^{-1} f(u).$$

A tier Θ specifies which symbols are relevant for determining future output behavior. $\mathrm{Suff}^1_\Theta(w)$ is the last symbol of w on Θ tier. The distinction between the input and output-based analyses is where this context is taken from:

2-OTSL	The most recent symbol on the *output tier*: $\mathrm{Suff}^1_\Theta(f^p(u))$, where $\Theta \subseteq \Delta$.
2-ITSL	The most recent symbol on the *input tier*: $\mathrm{Suff}^1_\Theta(u)$, where $\Theta \subseteq \Sigma$.

In both cases, two input prefixes with the same relevant tier suffix must have the same residual output behavior: any continuation must produce the same additional output, once the output already determined by each prefix is removed. Burness and McMullin (2019) show that a 2-OTSL function has a unique *canonical tier*: the largest tier satisfying this condition. An analogous result holds for 2-ITSL functions, since the union of two valid 2-tiers is also valid. We can therefore begin with the full input alphabet and eliminate symbols that violate the required locality condition.

When can a tier symbol be removed? Suppose two input prefixes u and v have the same most recent symbol t on the current candidate tier Ω. If appending the same input symbol x produces different output contributions, then t cannot serve as their shared context:

$$\mathrm{Suff}^1_\Omega(u) = \mathrm{Suff}^1_\Omega(v) = t, \qquad \mathrm{cont}_f(u,x) \neq \mathrm{cont}_f(v,x),$$

where $x \in \Sigma \cup \{\ltimes\}$.
We therefore remove t from the candidate tier. The crucial adaptation from the Burness and McMullin (2019)'s output-based tier learning is that prefixes are grouped by their most recent *input-tier* symbol rather than their most recent *output-tier* symbol.

5 CanTIER-2: inferring input-tier content

The elimination criterion introduced above provides the basis for CanTIER-2. Given a finite sample of input–output pairs $S \subseteq \Sigma^* \times \Delta^*$, the learner first estimates the prefix function and then compares output contributions to identify symbols that can be removed from the input tier.

Algorithm 1: PFE (prefix-function estimation)
The set $\mathrm{Pref}^*(w)$ contains all valid prefixes of w.

$$\begin{aligned} X &\leftarrow \bigcup_{(w,z)\in S} \mathrm{Pref}^*(w), \\ Y &\leftarrow \{y \in X : \ \forall a \in \Sigma,\ ya \in X\}, \\ P &\leftarrow \{(y, \mathrm{lcp}\{z : (w,z) \in S,\ y \in \mathrm{Pref}^*(w)\}) : y \in Y\}. \end{aligned}$$

Algorithm 2: CanTIER-2 (input-tier inference)

1. Initialize $\Theta \leftarrow \Sigma$ and compute $P = \mathrm{PFE}(S)$.
2. For each $t \in \Theta$, collect the prefix pairs whose most recent input-tier symbol is t:
$$M_t = \{(y,z) \in P : \mathrm{Suff}^1_\Theta(y) = t\}.$$
3. For each $x \in \Sigma$, compare the output contributions associated with the same continuation:
$$C_{t,x} = \{z^{-1}z' : (y,z) \in M_t,\ (yx, z') \in P\}.$$
If $|C_{t,x}| > 1$, mark t for removal. Apply the same comparison to word-final contributions.
4. Remove all marked symbols from Θ and repeat until no further symbols can be eliminated.

PFE retains input prefixes for which every possible one-symbol continuation occurs in the sample. Each retained prefix is paired with the longest common prefix of the corresponding observed outputs.

CanTIER-2 then starts with the full input alphabet and removes symbols when prefixes sharing the same last tier symbol produce different contributions under the same continuation.

The requirement imposed by PFE is demanding: every one-symbol extension must be represented in the sample. Such coverage is difficult to obtain from small phonological datasets, particularly when phonotactic restrictions limit possible continuations.

6 Sparse-data inference: PFEr and CanTIERr-2

We relax the PFE valid prefix requirement: in PFE*r*, a prefix is eligible if at least one one-symbol extension occurs in the sample. We also relax CanTIER-2 to accommodate sparse data, yielding CanTIERr-2. The relaxed algorithms are given below.

Algorithm 3: PFEr (relaxed prefix-function estimation)
As in PFE, $\mathrm{Pref}^*(w)$ denotes the set of all prefixes of w. The expression $\mathrm{Pref}_n(w)$ denotes the prefix of w of length n.

$$
\begin{aligned}
X &\leftarrow \bigcup_{(w,z)\in S} \mathrm{Pref}^*(w),\\
Y_r &\leftarrow \{y \in X : \exists a \in \Sigma,\ ya \in X\},\\
L_y &\leftarrow \mathrm{lcp}\{z : (w,z) \in S,\ y \in \mathrm{Pref}^*(w)\}, \qquad y \in Y_r,\\
P_r &\leftarrow \left\{\left(y, \mathrm{Pref}_{\min(|y|,|L_y|)}(L_y)\right) : y \in Y_r\right\}.
\end{aligned}
$$

Algorithm 4: CanTIERr-2 (relaxed input-tier inference)

1. Initialize $\Theta \leftarrow \Sigma$ and compute $P_r = \mathrm{PFEr}(S)$.
2. For each $t \in \Theta$, collect the input prefixes whose most recent tier symbol is t:
$$M_t = \{(y,z) \in P_r : \mathrm{Suff}^1_\Theta(y) = t\}.$$
3. For each attested next input symbol x, collect the corresponding output contributions:
$$W_{t,x} = \{(y, \mathrm{first}(z^{-1}z')) : (y,z) \in M_t,\ (yx, z') \in P_r\}.$$
4. Mark t for removal if there are $(y_1,r_1),(y_2,r_2) \in W_{t,x}$ such that $y_1 \neq y_2$ and $r_1 \neq r_2$. Apply the same test to word-final contributions:
$$W_{t,\ltimes} = \{(y, \mathrm{first}(z^{-1}u)) : (y,z) \in M_t,\ (y,u) \in S\}.$$
5. Remove all marked symbols from Θ and repeat until no further symbols can be eliminated.

Here $\mathrm{first}(r)$ denotes the first symbol of r, with $\mathrm{first}(\lambda) = \lambda$.

The main relaxation is that PFEr allows contributions to be estimated from input prefixes with only one attested continuation. However, a single input prefix cannot provide evidence for tier deletion. CanTIERr-2 therefore explicitly requires two distinct input prefixes with the same tier context to yield different contributions under the same continuation. This requirement is already implicit in the original algorithm, but becomes particularly relevant when prefixes are admitted on the basis of sparse data.

The relaxed algorithms do not retain the original convergence guarantees. When the available data do not provide sufficient evidence for deletion, the learner conservatively retains additional symbols on the tier.

7 Turkish experiment: learned tier content

We first evaluate CanTIERr-2 on the Turkish input–output data of Ellis et al. (2022), comprising 265 pairs. Forms involving /j/-epenthesis are excluded to separate the learning targets. The learner is supplied with a feature projection Φ, a satisfactory feature bundle to predict a target feature ϕ, with $k = 2$ fixed. The experiment concerns *tier-content inference*, rather than joint learning of the relevant features, tier, and output mapping. Future work will incorporate all those components together.

7.1 Feature representation

The feature representation determines which distinctions are available to the learner. The Turkish experiments use [TAP] to distinguish /j/ from vowels: in the adopted feature system, [CONS] does not reliably distinguish /i/ from /j/. The value 0 indicates that a feature is unspecified or inapplicable. The table gives representative feature values in the input inventory, *not* the learned tier.

	/j/	/o/	/u/	/i/	/ɯ/	/y/	/e/	/ɑ/	*I*	*E*	/b/	/k/
back	−	+	+	−	+	−	−	+	0	0	0	0
front	+	−	−	+	−	+	+	−	0	0	0	0
round	−	+	+	−	−	+	−	−	0	−	−	−
low	−	−	−	−	−	−	−	+	−	0	0	−
tap	−	0	0	0	0	0	0	0	0	0	−	−
high	+	−	+	+	+	+	−	−	+	−	0	+

7.2 Learned tiers by target feature

Each learned tier Θ consists of *feature bundles*, represented below as matrices in the order specified by Φ. The target feature being learned (ϕ) appears in boldface. The segment labels identify the members of the input inventory represented by each bundle; they are not separate symbols supplied to the learner.

Supplied Φ	Retained bundles in Θ (represented segments)		
$\begin{bmatrix}\textbf{back}\\ \text{tap}\end{bmatrix}$	$\begin{bmatrix}+\\0\end{bmatrix}$ back V	$\begin{bmatrix}-\\0\end{bmatrix}$ front V	$\begin{bmatrix}-\\-\end{bmatrix}$ /j/
	$\begin{bmatrix}0\\0\end{bmatrix}$ E, I		
$\begin{bmatrix}\textbf{front}\\ \text{tap}\end{bmatrix}$	$\begin{bmatrix}+\\0\end{bmatrix}$ front V	$\begin{bmatrix}-\\0\end{bmatrix}$ back V	$\begin{bmatrix}+\\-\end{bmatrix}$ /j/
	$\begin{bmatrix}0\\0\end{bmatrix}$ E, I		
$\begin{bmatrix}\textbf{low}\\ \text{back}\\ \text{tap}\end{bmatrix}$	$\begin{bmatrix}+\\+\\0\end{bmatrix}$ /ɑ/	$\begin{bmatrix}-\\+\\0\end{bmatrix}$ /o,u,ɯ/	$\begin{bmatrix}-\\-\\-\end{bmatrix}$ /j/
	$\begin{bmatrix}-\\-\\0\end{bmatrix}$ /i,y,e/	$\begin{bmatrix}0\\0\\0\end{bmatrix}$ E	
$\begin{bmatrix}\textbf{round}\\ \text{back}\\ \text{tap}\\ \text{high}\end{bmatrix}$	$\begin{bmatrix}+\\+\\0\\+\end{bmatrix}$ /u/	$\begin{bmatrix}+\\+\\0\\-\end{bmatrix}$ /o/	$\begin{bmatrix}+\\-\\0\\+\end{bmatrix}$ /y/
	$\begin{bmatrix}-\\+\\0\\+\end{bmatrix}$ /ɯ/	$\begin{bmatrix}-\\+\\0\\-\end{bmatrix}$ /ɑ/	$\begin{bmatrix}-\\-\\-\\+\end{bmatrix}$ /j/
	$\begin{bmatrix}-\\-\\0\\+\end{bmatrix}$ /i/	$\begin{bmatrix}-\\-\\0\\-\end{bmatrix}$ /e/	$\begin{bmatrix}-\\0\\0\\-\end{bmatrix}$ E
	$\begin{bmatrix}0\\0\\0\\+\end{bmatrix}$ I		

For [BACK] and [FRONT], the learner eliminates the transparent consonant bundle, but retains /j/ and the bundle representing the underspecified suffix vowels E and I. This does not imply that /j/ triggers harmony. Although /j/ occurs in 30 inputs, only four can provide relevant contexts after the other consonants are removed from the tier. All four contain a preceding back vowel, as in /kojnI/ $\mapsto$ [kojnu] and /kojnE/ $\mapsto$ [kojnɑ]. Without corresponding front-vowel contexts, the sample does not supply a contradiction that would eliminate /j/. Likewise, no attested underspecified vowel is followed by another underspecified vowel, so the learner retains that bundle.

For [LOW], the consonant bundles and I are removed, while E, the nonhigh underspecified suffix vowel, remains. For [ROUND], adding [HIGH] produces a finer partition of the vowels, while the feature projection reduces the consonant inventory to a small number of bundles, which are eliminated.

The learned tiers are thus conservative: they retain bundles that the available data do not justify deleting. Consequently, the learned tiers may be larger than those expected in a typical phonological analysis. This may reflect both the limited sample and Turkish phonotactic restrictions, which may prevent the occurrence of contexts needed to establish the relevant contradictions.

8 From the learned tier to an input transducer

The learned tier determines the structure of an input-oriented transducer. Consider the [BACK] projection from the Turkish experiment. We abbreviate the four retained feature bundles as

$$F = \begin{bmatrix}-\\0\end{bmatrix}, \quad B = \begin{bmatrix}+\\0\end{bmatrix}, \quad J = \begin{bmatrix}-\\-\end{bmatrix}, \quad U = \begin{bmatrix}0\\0\end{bmatrix},$$

with the consonant bundle $C = \begin{bmatrix}0\\-\end{bmatrix}$ excluded from the tier.

The states will correspond to the elements of the learned tier, F, B, J, and U. Reading an on-tier element updates the state, whereas an off-tier element never causes state-change. Although J and U remain on the learned tier, this does not imply that they trigger harmony.

Given the transducer structure in Figure 3, the output labels for individual features can be learned with an input-oriented learner, such as SEAL.

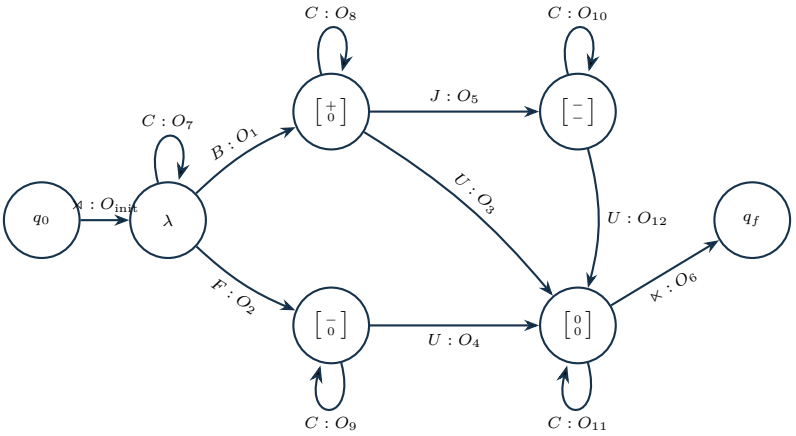


Figure 3: Input-oriented transducer skeleton based on the learned Turkish [BACK] tier. The figure shows representative transitions including the off-tier elements represented with self-loops in non-initial and non-final states. The O_i are the encoded outputs to be inferred.

9 Further experiments: learned input tiers

We also evaluate CanTIERr-2 on Kikuria, Finnish, and Yakut. As in the Turkish experiment, $k = 2$ is fixed, and the learner is supplied with a feature projection Φ and a target output feature ϕ.

Kikuria: regressive height harmony. The relevant feature projection and learned tier are

$$\Phi = \begin{bmatrix} \textbf{high} \\ \text{cons} \\ \text{low} \end{bmatrix}, \qquad \Theta = \left\{ \begin{bmatrix} + \\ - \end{bmatrix}, \begin{bmatrix} - \\ - \\ + \end{bmatrix}, \begin{bmatrix} - \\ - \\ - \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \end{bmatrix} \right\}.$$

The first three bundles represent the high vowels /i,u/, the low vowel /a/, and the mid vowels /e,o/, respectively, including their long counterparts. Applicative /e/ triggers lowering of preceding high vowels, while /a/ can block the process, as illustrated by /suraaŋgera/, where /u/ remains high despite the following /e/ in the suffix. The additional bundle represents /h/, which is retained because the sample does not provide sufficient evidence for its removal. The experiment uses 32 normalized input–output pairs (24 distinct pairs after feature projection), with long vowels represented as single segments.

Finnish: progressive backness harmony. The supplied projection and learned tier are

$$\Phi = \begin{bmatrix} \textbf{back} \\ \text{syll} \\ \text{low} \\ \text{round} \end{bmatrix}, \qquad \Theta = \left\{ \begin{bmatrix} + \\ + \\ + \\ - \end{bmatrix}, \begin{bmatrix} + \\ + \\ - \\ + \end{bmatrix}, \begin{bmatrix} - \\ + \\ + \\ - \end{bmatrix}, \begin{bmatrix} - \\ + \\ - \\ + \end{bmatrix}, \begin{bmatrix} - \\ - \\ - \\ - \end{bmatrix} \right\}.$$

The first four bundles represent the harmonic vowel classes /ɑ/, /o,u/, /æ/, and /y,ø/. The neutral vowels /e,i/ are excluded from the tier, while the final bundle, /j/, is conservatively retained. The separate word-final, local /e/→[i] alternation is not accounted for.

Yakut: progressive backness harmony. For backness, the learner receives

$$\Phi = \begin{bmatrix} \textbf{back} \\ \text{syll} \end{bmatrix}, \qquad \Theta = \left\{ \begin{bmatrix} + \\ + \end{bmatrix}, \begin{bmatrix} - \\ + \end{bmatrix}, \begin{bmatrix} 0 \\ + \end{bmatrix} \right\}.$$

The three bundles represent back vowels, front vowels, and underspecified suffix vowels (/A,I:/). All consonant bundles are eliminated.

Yakut: progressive rounding harmony. For rounding, the supplied projection and learned tier are

$$\Phi = \begin{bmatrix} \textbf{round} \\ \text{high} \\ \text{syll} \end{bmatrix},$$

$$\Theta = \left\{ \begin{bmatrix} + \\ + \\ + \end{bmatrix}, \begin{bmatrix} + \\ - \\ + \end{bmatrix}, \begin{bmatrix} - \\ + \\ + \end{bmatrix}, \begin{bmatrix} - \\ - \\ + \end{bmatrix}, \begin{bmatrix} 0 \\ + \\ + \end{bmatrix}, \begin{bmatrix} 0 \\ - \\ + \end{bmatrix}, \begin{bmatrix} - \\ - \\ - \end{bmatrix} \right\}.$$

The first four bundles distinguish rounded high /u,y/, rounded nonhigh /ɔ,œ/, unrounded high /i,ɯ/, and unrounded nonhigh /ɑ,ɛ/ vowels. The next two represent the underspecified suffix vowels /I:/ and /A/. The final bundle represents /χ/, a consonant retained because the available data do not justify its removal.

High suffix vowels undergo rounding harmony after rounded vowels of either height, whereas nonhigh suffix vowels round only after nonhigh rounded vowels. The distinction encoded by [HIGH] is therefore relevant to the ROUND mapping.

Across these experiments, the learner eliminates transparent elements while retaining bundles for which the sample provides insufficient evidence for deletion. The resulting tiers are conservative and need not (sadly) overlap with the smallest linguistically motivated tiers.

10 What the results establish—and open questions

The Turkish analysis shows that vowel harmony, although describable as an output-oriented process, also admits a tier-based definite input-oriented analysis. CanTIERr-2 learns input-tier content from sparse, feature-projected data, determining the input-state structure of a transducer whose output labels can then be inferred separately by input-based learner. The experiments on Kikuria, Finnish, and Yakut suggest that this approach extends to other harmony processes.

1. **Learning feature combinations.** The current experiments assume that the relevant input feature projection Φ and target output feature ϕ are supplied in advance. An open question is whether the learner can identify the relevant feature combinations directly from the data (as has been done in learning local processes), rather than relying on a predetermined representation.

2. **Phonotactics and conservative tier inference.** The learner may retain elements that are phonologically transparent because the sample does not provide the contradictory contexts needed to eliminate them. This may reflect not only data sparsity but also phonotactic restrictions that prevent certain contexts from ever occurring. An open question is how such restrictions should be incorporated into learning. Should the learner first infer the phonotactics of the language and learn the mapping over well-formed inputs, or should it aim to learn a total function over the full input alphabet?

3. **Combining feature-specific transducers.** The current approach learns tiers and output mappings for individual features. Different features may require different input tiers, resulting in transducers with different state structures. Although feature-specific transducers can be combined for local processes, tier-based dependencies introduce an additional challenge: a symbol may update the context for one feature while remaining transparent to another. An open question is how to combine these transducers into a single segmental mapping, or, more broadly, at which stages of learning feature-based representations are beneficial and when they may complicate inference.

4. **Learning multiple phonological processes.** The current experiments focus on individual harmony processes. However, a language may exhibit several processes simultaneously, as in Kera, where both local and non-local processes interact. An open question is how the learner can identify these different dependencies and determine whether they should be learned separately or as part of a single mapping.

References

Burness, Phillip, and Kevin McMullin. 2019. Efficient learning of Output Tier-based Strictly 2-Local functions. *Proceedings of the 16th Meeting on the Mathematics of Language*, 78–90.

Burness, Phillip, Kevin McMullin, and Jane Chandlee. 2021. Long-distance phonological processes as tier-based strictly local functions. *Glossa* 6(1):1–37.

Chandlee, Jane, Rémi Eyraud, and Jeffrey Heinz. 2015. Output Strictly Local Functions. *Proceedings of the 14th Meeting on the Mathematics of Language*, 112–125.

Chandlee, Jane, and Jeffrey Heinz. 2018. Strict locality and phonological maps. *Linguistic Inquiry* 49(1):23–60.

Ellis, Kevin, Adam Albright, Armando Solar-Lezama, Joshua B. Tenenbaum, and Timothy J. O'Donnell. 2022. Synthesizing theories of human language with Bayesian program induction. *Nature Communications* 13:5024.

Lambert, Dakotah. 2023. Relativized adjacency. *Journal of Logic, Language and Information* 32(4):707–731.

Lambert, Dakotah, and Jeffrey Heinz. 2023. An algebraic characterization of total input strictly local functions. *Proceedings of the Society for Computation in Linguistics* 6:25–34.

Lambert, Dakotah, and Jeffrey Heinz. 2024. Algebraic reanalysis of phonological processes described as output-oriented. *Proceedings of the Society for Computation in Linguistics* 7:129–138.

Markowska, Magdalena, and Jeffrey Heinz. 2026. Learning k-ISL functions with phonological features. *Journal of Language Modelling* 14(1):1–80.

Odden, David. 2005. *Introducing Phonology*. Cambridge University Press.